



AI AGENT WORKFLOWS WITH AZURE FUNCTIONS (AND FRIENDS)

Cincy Deliver 2026 Sponsors

Diamond



Platinum



Gold





MICHAEL S. COLLIER

HUSBAND, FATHER, BUCKEYE, GOLFER,
BBQ, & MUSTANGS

*PRINCIPAL ARCHITECT
@ CENTRIC CONSULTING*



[HTTPS://MICHAELSCOLLIER.COM](https://MICHAELSCOLLIER.COM)





FOLLOW ALONG



DURABILITY

NEW TECHNOLOGY . . . SIMILAR PROBLEMS

MICROSOFT AGENT FRAMEWORK

Microsoft's multi-language framework for building, orchestrating & deploying AI Agents

Successor to Semantic Kernel and AutoGen.

Framework for agent orchestration. Supports multiple providers

Providers – the inference service that powers the agent (Azure OpenAI, Ollama, Anthropic, etc.)

Simple chat agents → complex, multi-agent workflows

Core capabilities:

- **Agents:** use LLMs to process inputs, call tools and MCP servers, and generate responses
- **Workflows:** graph-based workflows that connect agents and functions for multi-step tasks with type-safe routing, checkpointing, and human-in-the-loop



MICROSOFT AGENT FRAMEWORK

```
using System;
using Azure.AI.OpenAI;
using Azure.Identity;
using Microsoft.Agents.AI;

AIAgent agent = new AzureOpenAIClient(
    new
    Uri(Environment.GetEnvironmentVariable("AZURE_OPENAI_ENDPOINT")),
    new AzureCliCredential())
    .GetChatClient("gpt-5-mini")
    .AsAIAgent(instructions: "You're an expert on the movie, The Matrix.
    Answer all questions as Agent Smith.");

Console.WriteLine(await agent.RunAsync("Who was the star of the movie?"));
```

Add tools, middleware, workflows, etc. to build production applications.



WORKFLOWS

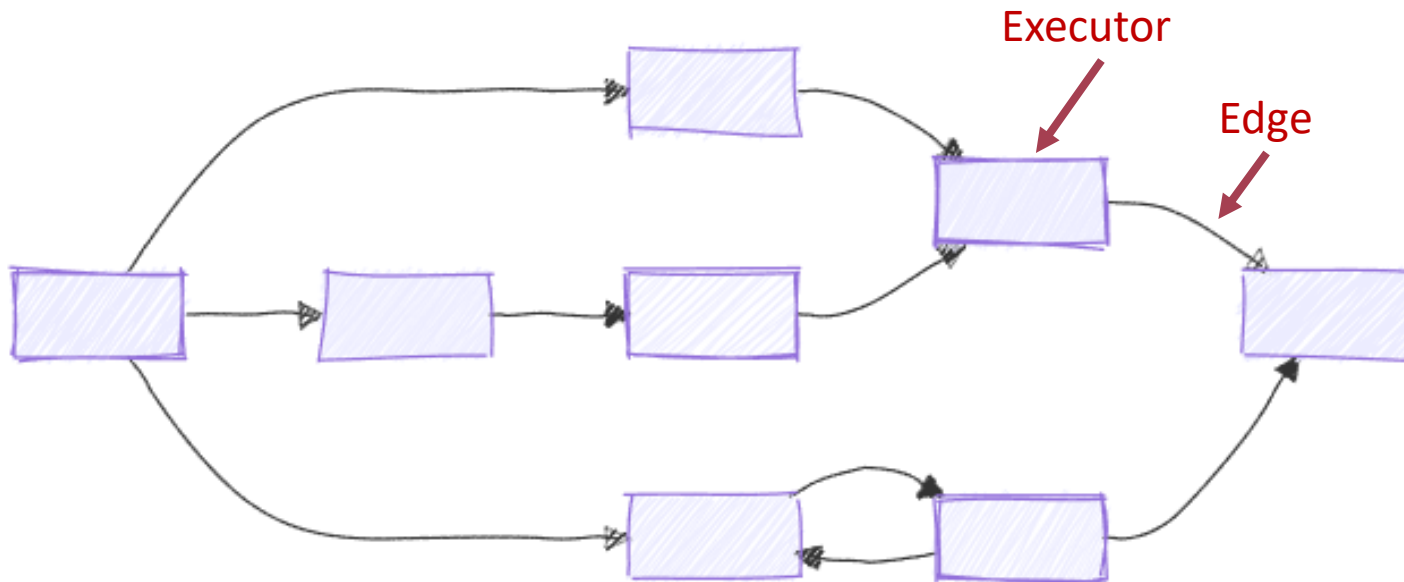
- When a single agent isn't enough.
- Multiple agents with different responsibilities, models, etc.



[This Photo](#) by Unknown Author is licensed under [CC BY-NC-ND](#)

WORKFLOWS

- When a single agent isn't enough.
- Multiple agents with different responsibilities, models, etc.
 - Order processing
 - Customer support
 - Human interaction



[This Photo](#) by Unknown Author is licensed under [CC BY-NC-ND](#)

- Define your agents.
- Build your workflow.

```
var workflow = new WorkflowBuilder(orderIntakeAgent)
    .AddEdge(orderIntakeAgent, fulfillmentAgent)
    .AddEdge(fulfillmentAgent, customerMessagingAgent)
    .Build();
```

- Workflow as Agents
- Unified interface
- Building block in larger systems

```
var workflow = new WorkflowBuilder(orderIntakeAgent)
    .AddEdge(orderIntakeAgent, fulfillmentAgent)
    .AddEdge(fulfillmentAgent, customerMessagingAgent)
    .Build()
    .AsAIAgent();
```

- Built-in Orchestration
 - Sequential, Concurrent
 - Handoff, Group Chat
 - Magentic*

```
var workflow = AgentWorkflowBuilder.BuildSequential([
    orderIntakeAgent,
    fulfillmentAgent,
    customerMessagingAgent
]);
```

DEMO

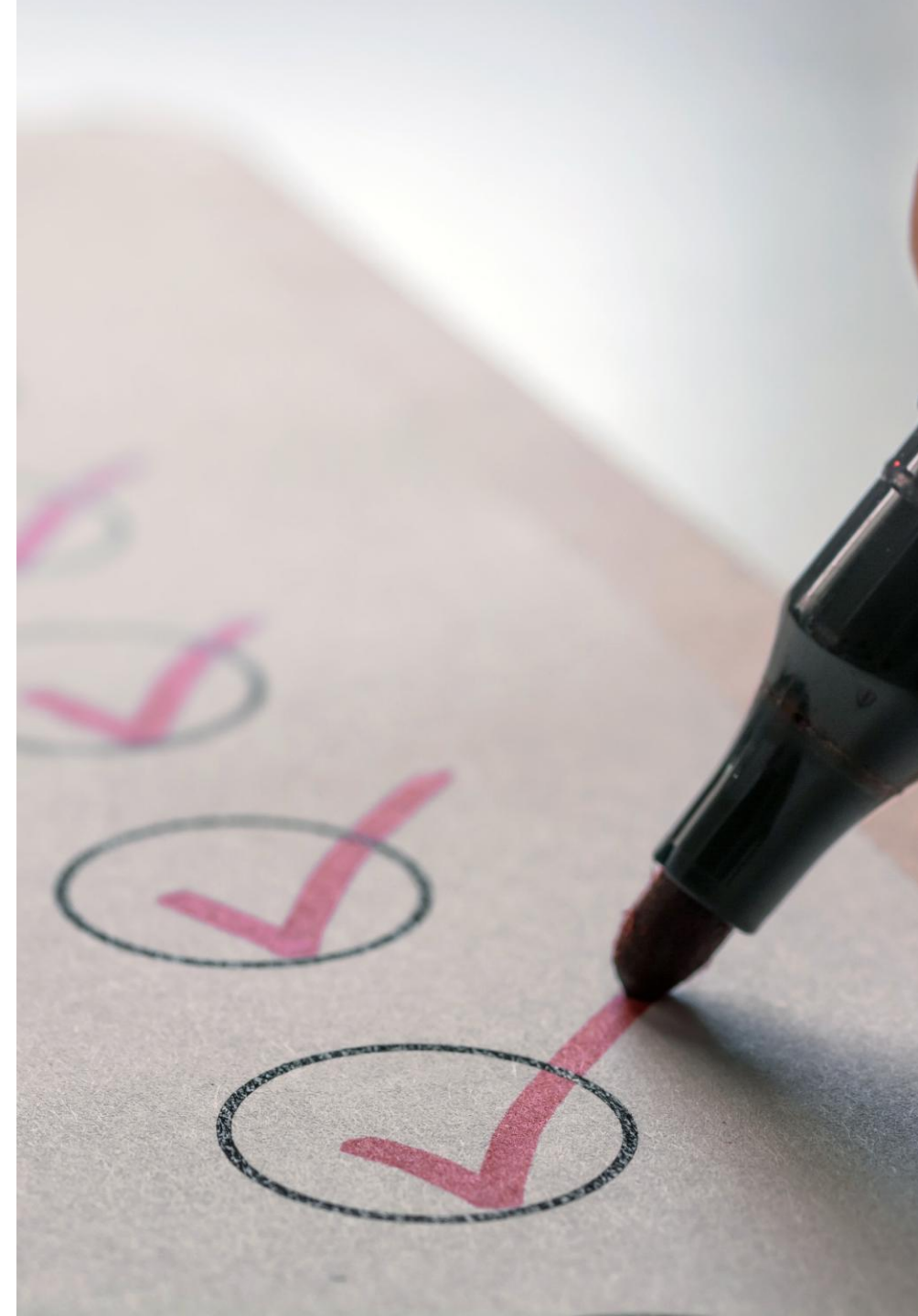
SIMPLE AGENT AND WORKFLOW



CHECKPOINTS

Ability to persist workflow state at a point in time

- The current state of all executors
- All pending messages in the workflow for the next superstep
- Pending requests and responses
- Shared states



MICROSOFT AGENT FRAMEWORK – CHECKPOINTS

```
CheckpointManager checkpointManager = CheckpointManager.CreateJson(new
FileSystemJsonCheckpointStore(new DirectoryInfo("./checkpointstore")));

// workflow
await using StreamingRun run = await InProcessExecution.RunStreamingAsync(workflow,
    new ChatMessage(ChatRole.User, input),
    checkpointManager: checkpointManager);
```

MICROSOFT AGENT

```
CheckpointManager checkpointManager  
FileSystemJsonCheckpointStore(new [
```

```
// workflow
```

```
await using StreamingRun run = await  
    new ChatMessage(ChatRole.User,  
        checkpointManager: checkpointMa
```

```
{  
  "stepNumber": 0,  
  "workflow": {  
    "executors": { ...  
    },  
    "edges": {  
      "OrderIntakeAgent_c8f8cdba79be4e399ca612076e871549": [ ...  
    ],  
      "FulfillmentAgent_addce6a81d5d4da39a831bc6be83aa26": [ ...  
    ]  
    }  
  },  
  "requestPorts": [],  
  "startExecutorId": "OrderIntakeAgent_c8f8cdba79be4e399ca612076e871549",  
  "outputExecutorIds": []  
},  
"runnerData": {  
  "instantiatedExecutors": [ ...  
  ],  
  "queuedMessages": {  
    "FulfillmentAgent_addce6a81d5d4da39a831bc6be83aa26": [  
      {  
        "messageType": { ...  
      },  
      "message": {  
        "typeId": { ...  
      },  
      "value": [  
        {  
          "role": "user",  
          "contents": [  
            {  
              "$type": "text",  
              "text": "my name is michael collier and i would like 2 units of vanilla exception"  
            }  
          ]  
        }  
      ]  
    }  
  ],  
  "source": {  
    "id": "OrderIntakeAgent_c8f8cdba79be4e399ca612076e871549"  
  }  
},  
},  
"messageType": { ...  
},  
"message": {
```

MICROSOFT AGENT FRAMEWORK – CHECKPOINTS

Developers need to opt-in.

- Create a `CheckpointManager`
 - `FileSystemJsonCheckpointStore`
 - `CosmosCheckpointStore`
- Determine when and how to resume or rehydrate from saved checkpoints, and which specific checkpoint.

Doable – difficult to get right at scale!

Missing piece – durability and automatic recovery



AGENT FRAMEWORK + AZURE DURABLE FUNCTIONS == DURABLE AGENTS

Build stateful AI agents & multi-agent orchestrations in a serverless environment

- Persist state automatically & resume after failures
- On-demand automatic scaling
- Orchestrate multi-agent workflows with reliable execution guarantees
- View agent inputs & outputs (including tools)

Durability as a hosting concern rather than redesigning the agent's control flow

When?

- Full code control – deploy & manage compute environment
- Complex orchestrations – multiple agents w/ deterministic long-running workflows
- Event driven orchestrations – Azure Functions triggers (HTTP, Service Bus, Storage, etc.)
- Automatic conversation state – automatic management w/o requiring code

Preview

DURABLE AGENTS

```
AIAgent agent = new AzureOpenAIClient(  
    new Uri(Environment.GetEnvironmentVariable("AZURE_OPENAI_ENDPOINT")),  
    new AzureCliCredential())  
    .GetChatClient("gpt-5-mini")  
    .AsAIAgent(instructions: "You're an expert on the movie, The Matrix. Answer all  
        questions as Agent Smith.");  
  
using IHost app = FunctionsApplication  
    .CreateBuilder(args)  
    .ConfigureFunctionsWebApplication()  
    .ConfigureDurableAgents(options => options.AddAIAgent(agent))  
    .Build();  
app.Run();  
  
// Use the Durable Agent  
DurableAIAgent customerServiceAgent = context.GetAgent("CustomerServiceAgent");
```

DURABLE WORKFLOW

```
Workflow orderProcessingWorkflow = new WorkflowBuilder(orderIntakeAgent)
    .WithName("order-processing-workflow")
    .WithDescription("Workflow to process customer orders")
    .AddEdge(orderIntakeAgent, fulfillmentDecisionAgent)
    .AddEdge(fulfillmentDecisionAgent,
customerMessagingAgent)
    .Build();
```

```
builder.ConfigureDurableWorkflows(workflows =>
{
    workflows.AddWorkflow(orderProcessingWorkflow);
});
```

DURABLE WORKFLOW

- Async HTTP call by default
- Need a synchronous response? Use **x-ms-wait-for-response** HTTP request header.

```
var client = httpClientFactory.CreateClient("self");
using var content = JsonContent.Create(order, options: JsonOptions);
using var response = await client.PostAsync(
    "api/workflows/order-processing-workflow/run", content, cancellationToken);

if (!response.IsSuccessStatusCode)
{
    // Handle the error
}
```

DURABLE TASK SCHEDULER

DTS is a purpose-built fully-managed Azure service for Durable Task Framework.

- Durable Functions built on Durable Task Framework.
- Dashboard to visualize orchestrations.
- Send events via dashboard.
- Aspire integration!

```
#pragma warning disable ASPIREDURABLETASK001

var dtsScheduler = builder.AddDurableTaskScheduler("scheduler");
if (builder.ExecutionContext.IsRunMode)
{
    dtsScheduler.RunAsEmulator();
}
var dtsTaskHub = dtsScheduler.AddTaskHub("default");

#pragma warning restore ASPIREDURABLETASK001
```



Durable Task Scheduler

TASK HUB

 Entities

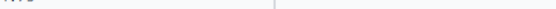
 Agents

Schedules

Workers & Metrics

Timeline History Flow

From First Activity

ACTIVITY	0ms	5.0s	10.0s	15.0s
1 Run → @dafx-customerservicea	 6.9s			
2 Run → @dafx-emailagent@95b	 4.7s			
3 SendCustomerEmailActivity	 6.5s			
4 ProcessFeedbackActivity	 2ms			

 Orchestration
  Activity
  Sub-Orchestration
  Timer
  Event
  Entity
  Completed
  Failed
  Running

ASPIRE RESOURCE ORCHESTRATION

Configure Azure Storage & DTS via local emulators.

Existing Azure resources:

- Azure Open AI resource
- Azure Communication Service
- Azure Service Bus

2 projects:

- Durable Functions orchestration (agents & workflows)
- Web application

Parameters & env vars defined in one place.

```
// Azure Durable Task Scheduler
#pragma warning disable ASPIREDURABLETASK001
var dtsScheduler = builder.AddDurableTaskScheduler("scheduler");
var dtsTaskHub = dtsScheduler.AddTaskHub("default");

#pragma warning restore ASPIREDURABLETASK001

var storage = builder.AddAzureStorage("storage");

var sb = builder.AddAzureServiceBus("messaging")
    .AsExisting(serviceBusName, serviceBusResourceGroup);

_ = sb.AddServiceBusQueue(feedbackQueueName);
_ = sb.AddServiceBusQueue(orderQueueName);

var func = builder.AddAzureFunctionsProject<Projects.DurableAgent_Functions>("func")
    .WithHostStorage(storage)
    .WithReference(dtsTaskHub)
    .WithReference(sb)
    .WithEnvironment("AZURE_OPENAI_ENDPOINT", azureOpenAIEndpoint)
    .WithEnvironment("AZURE_OPENAI_DEPLOYMENT", azureOpenAIDeployment)
    .WithEnvironment("FEEDBACK_QUEUE_NAME", feedbackQueueName)
    .WithEnvironment("ORDER_QUEUE_NAME", orderQueueName)
    .WithEnvironment("APPLICATIONINSIGHTS_CONNECTION_STRING", applicationInsightsConnectionString)
    .WithEnvironment("OTEL_SOURCE_NAME", "DurableAgentDemo")
    .WithEnvironment("OTEL_SERVICE_NAME", "DurableAgentService")
    .WithEnvironment("DOTNET_ENVIRONMENT", "Development")
    .WithEnvironment("RECIPIENT_EMAIL_ADDRESS", recipientEmailAddress)
    .WithEnvironment("SENDER_EMAIL_ADDRESS", senderEmailAddress)
    .WithEnvironment("EMAIL_SERVICE_ENDPOINT", emailServiceEndpoint)
    .WithExternalHttpEndpoints()
    .WaitFor(storage)
    .WaitFor(dtsScheduler)
    .WaitFor(sb);

func.WithReference(func);

var web = builder.AddProject<Projects.DurableAgent_Web>("web")
    .WithReference(func)
    .WithExternalHttpEndpoints()
    .WaitFor(func);

builder.Build().Run();
```





Welcome to Froyo Foundry!

Where Every Scoop is `async` Delicious!

Fuel your coding sessions with frozen yogurt. We debug recipes and deploy flavor. Your feedback helps us iterate faster! 🚀

`git commit --feedback`

`npm install froyo`



Optimized Flavors

Crafted with precision like your favorite algorithm -

`O(delicious)`



Developer Experience

Your satisfaction is our highest priority. No breaking changes to your taste buds!

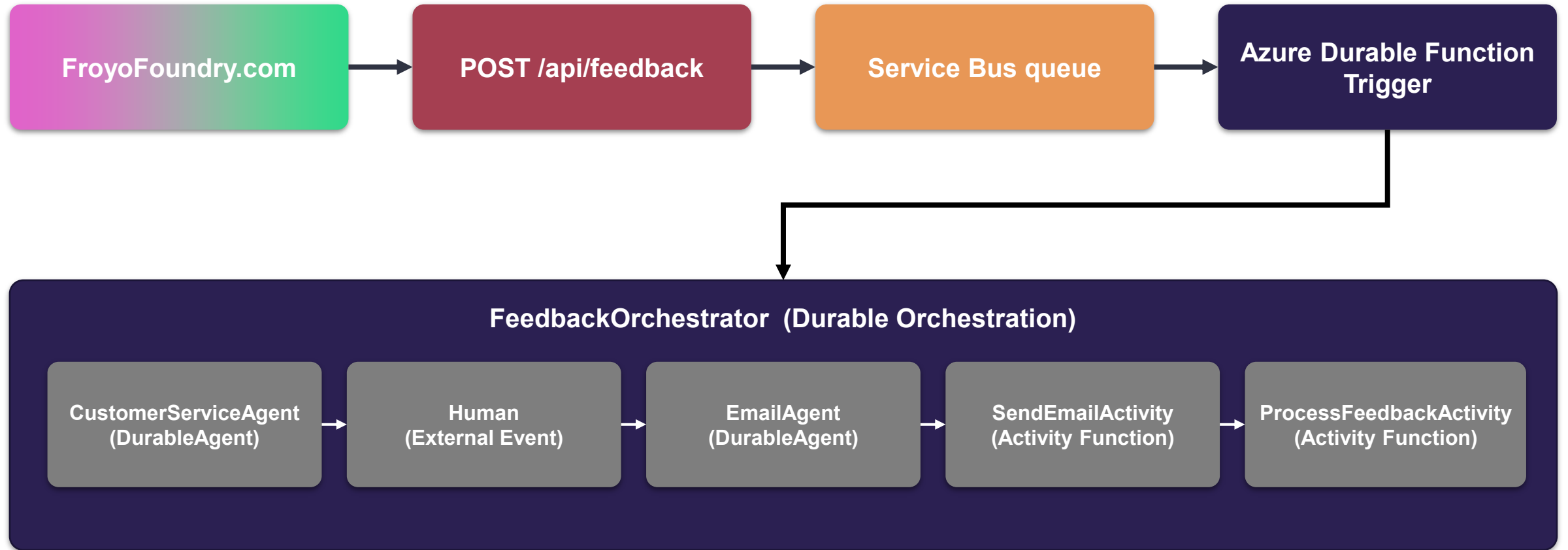


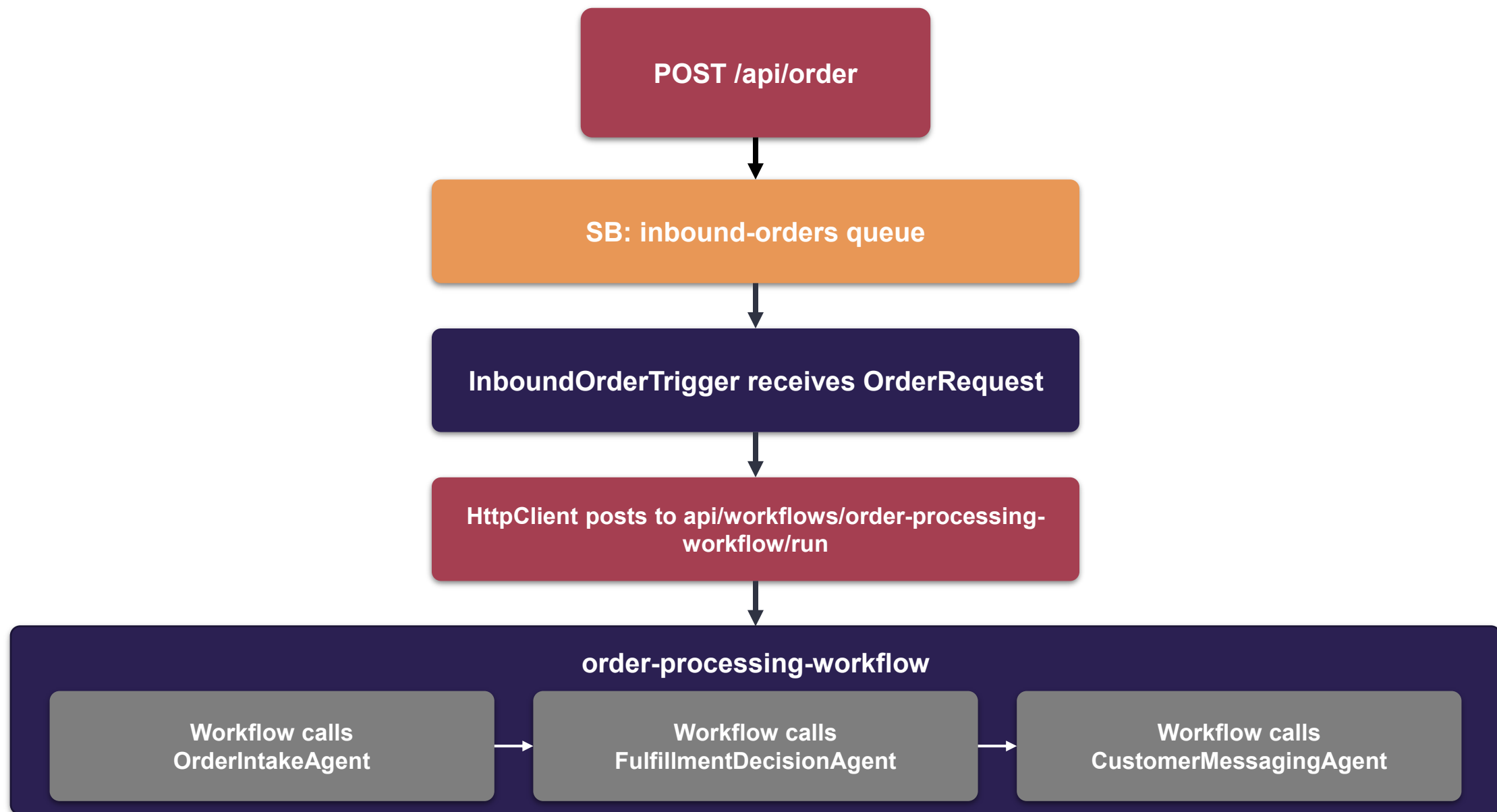
Low Latency Service

Fast serve times - because we know your `build` is still running



DEMO





RECAP



Agents are easy to build – reliability is hard.



State is the real problem!



Durability changes the execution model



Durable Agents & Workflows apply proven patterns



We're early, but the direction is clear!

RESOURCES

- <https://aka.ms/AgentFramework>
- [Checkpoints Are Not Durable Execution: Why LangGraph, CrewAI, Google ADK Fall Short | Diagrid Blog](#)
 - [Still Not Durable: How Microsoft Agent Framework and Strands Agents Repeat the Same Mistakes | Diagrid Blog](#)
- [Bulletproof agents with the durable task extension for Microsoft Agent Framework | Microsoft Community Hub](#)
- Durable Functions: <https://learn.microsoft.com/azure/azure-functions/durable/>
- Durable Task Scheduler: <https://learn.microsoft.com/azure/azure-functions/durable/durable-task-scheduler/durable-task-scheduler>

Demo code @

<https://github.com/mcollier/agent-framework-demos>

<https://github.com/mcollier/durable-agent-demo>



THANK YOU

Michael S. Collier

<https://michaelscollier.com>



Presentation

